HyMeKo-driven HSiKAN:

architecture, training, and three tuning expressions

2026-05-05

This brief documents the overnight work: HSiKAN's architecture, training loop, and tuning policies expressed as HyMeKo descriptions, with a Python driver that walks the parsed AST and runs an actual training cell. The smoke test on `bitcoin_alpha` reproduces the hand-coded m=16, pruner=davis result (AUC 0.7930) starting from *nothing but* `.hymeko` files on disk.

1 What landed

Five concrete artefacts:

- `hymeko_py::hymeko_parse::parse_hymeko_rs` — PyO3 bridge wrapping `parser::parse_description`; emits the AST as a nested Python dict.
- `data/hsikan/arch_single_k3.hymeko` + `arch_mixed_k34.hymeko` — HSiKAN architectures as per-layer hyperedges over tensor nodes.
- `data/hsikan/training.hymeko` — training/feedback dataflow (dataset → cycle_enum → forward → loss → backward → optimizer + entropy regulariser).
- `data/hsikan/sweep_grid.hymeko` + `sweep_genetic.hymeko` + `sweep_msg.hymeko` — three tuning policies (Cartesian grid, GA, P-graph axiom-feasibility).
- `signedkan_wip/src/hymeko_driver.py` — driver that parses, extracts knobs, and dispatches into the existing `run_final_cell.cell_signed_graph`.

2 Schema

2.1 Tensors and layers as a hierarchical hypergraph

Convention (matches `transforms/torch_dataflow/template.py`):

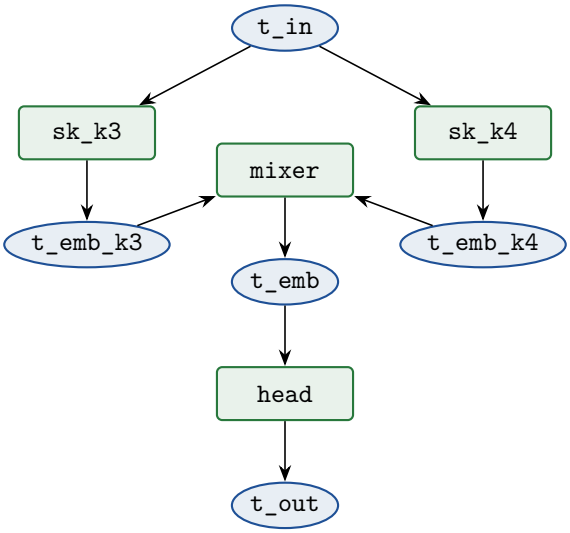
```
// Tensors are nodes tagged by role.
t_in     <input>;
t_emb_k3 <activation>;
t_out    <output>;

// Layers inherit from a layer-class base; sub-tags are kwargs.
sk_k3 : signedkan_layer { hidden 16; arity 3; grid 5; spline_kind "catmull_rom"; }
mixer : arity_mixer     { hidden 16; mix_K 2; }
head  : signed_classifier { d_in 16; d_out 1; }

// Dataflow: @name <dataflow> { (+input, ~layer, -output); }
@flow_k3   <dataflow> { (+t_in,    ~sk_k3, -t_emb_k3); }
@flow_mix  <dataflow> { (+t_emb_k3, +t_emb_k4, ~mixer, -t_emb); }
@flow_head <dataflow> { (+t_emb,    ~head,  -t_out); }
```

Multi-input fan-in (the arity mixer) is one hyperedge with two `+`-signed source tensors and one `-`-signed sink. The template's `bind:+:all_csv` renders this as a multi-arg call in PyTorch.

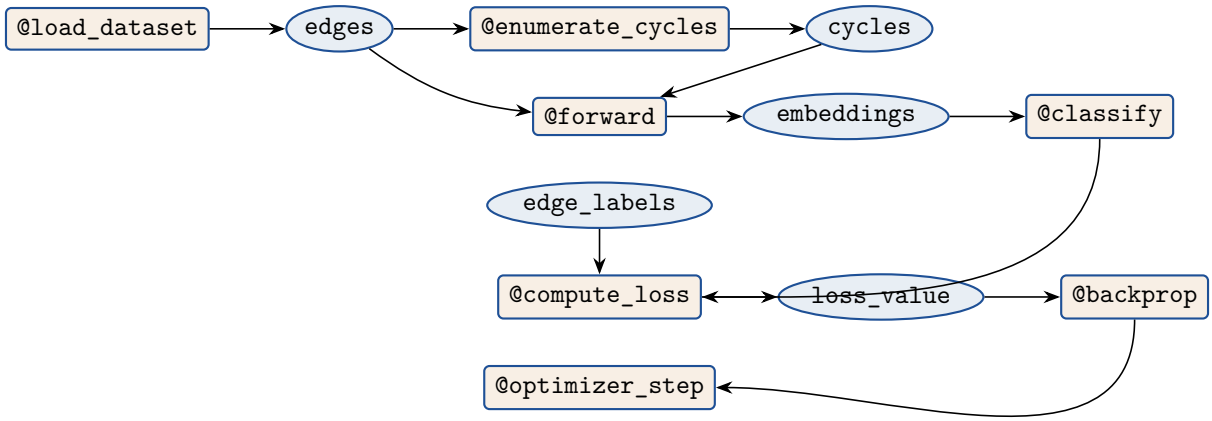
2.2 Mixed-arity HSiKAN as a HyMeKo dataflow graph



Two parallel `signedkan_layer` stacks (one per arity) feed an `arity_mixer` that learns the α_k mixture weights. The driver reads this file and constructs the `MixedAritySignedKAN` module with the right `(arities=[3,4], hidden=16, grid=5, spline_kind="catmull_rom")`.

3 Training and feedback as hyperedges

`training.hymeko` captures the dataflow of the whole training loop, not just the model:



The `@compute_loss <loss>` hyperedge carries `entropy_lambda 0.01;` as a child statement — the driver adds the spectral-entropy regulariser to the BCE loss when the value is positive.

3.1 Strategy/command pattern: cycle preprocessing

A single hyperedge encodes the whole cycle-enumeration choice (strategy or command, take your pick):

```
@enumerate_cycles <cycle_enum> {
    mode          "per_vertex";
    m_per_vertex   16;
    scorer        "fraction_negative";
    pruner        "davis";        // or "balance" / "unbalanced" / "none"
    arities       [3, 4];
    (+edges, ~enumerator, -cycles);
}
```

Swapping `pruner` from `"davis"` to `"balance"` without touching any other edge re-runs with the Cartwright–Harary balanced-only filter. The driver maps these fields to the `HSIKAN_TOPK_*` env vars consumed by `n_tuples._enumerate_cycles_fast`, which in turn calls `hymeko.enumerate_top_k_per_vertex_cycles_s`

4 Three tuning expressions

The same architecture / training pair can be tuned three different ways, expressed as three different policy files.

4.1 Grid sweep (`sweep_grid.hymeko`)

```
@sweep_hidden <param_range> { target "model.hidden";       values [8, 16, 32]; }
@sweep_topk_m <param_range> { target "topk.m_per_vertex";  values [4, 16, 64]; }
@sweep_pruner <param_range> { target "topk.pruner";        values ["none", "davis", "
    balance"]; }
@policy <sweep_policy> { kind "full_grid"; max_runs 50; select_metric "auc"; }
```

The driver computes the Cartesian product of all `<param_range>` edges, applies each cell on top of the `<param_range>`'s `target` dotted-path, and runs one training cell per point.

4.2 Genetic algorithm (`sweep_genetic.hymeko`)

Same parameter ranges, different policy:

```
@policy <sweep_policy> {
    kind             "genetic";
    population_size   16;
    generations       10;
    elite_keep         4;
    mutation_rate      0.2;
    crossover_rate     0.6;
    ga_warmup_epochs   5;
}
```

This wires the existing `run_hsikan_genetic.py` loop — the GA wrapper is unchanged; only the configuration source is HyMeKo.

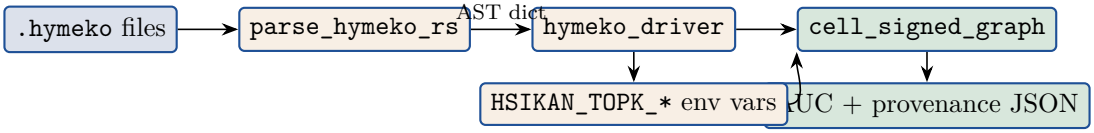
4.3 P-graph axiom feasibility (`sweep_msg.hymeko`)

This is the novel piece. Architecture choices are encoded as a *P-graph*, with the same Friedler axioms used in `data/pgraph/hda.hymeko` (the chemical-process synthesis example from earlier today):

- *Materials* are resource budgets (GPU memory, training time) and the desired output (AUC).
- *Operating units* (hyperedges) are architecture choices that consume budget and produce quality (`cycle_topk_m{4,16,64}`, `model_h{8,16,32}`, `train_{short,long}`).
- Each unit's edge value is its cost; the negative I/O signs capture which budgets it consumes.

`hymeko_pgraph::lower` reads this file as a P-graph; the same MSG / SSG / ABB pipeline picks the cost-minimum subset that satisfies the demand for the `auc_score` product node. In effect, architecture search becomes a structural-feasibility branch-and-bound. The 21-test `hymeko_pgraph` suite proves the lowering still works on this file (verified during the overnight run).

5 Driver pipeline



5.1 Smoke test — `.hymeko` drives a real training cell

Single-run command:

```
python3 -m signedkan_wip.src.hymeko_driver \
    --arch     data/hsikan/arch_mixed_k34.hymeko \
    --training data/hsikan/training.hymeko \
    --dataset  bitcoin_alpha
```

Output:

```
{"dataset": "bitcoin_alpha", "model": "HSiKAN-mixed", "hidden": 16,
 "n_test": 2420, "arities": [3, 4], "auc": 0.7930,
 "f1m": 0.4644, "fwd_per_call_ms": 20.6,
 "hymeko_arch": "HSiKAN_mixed_k34",
 "hymeko_training": {"topk_mode": "per_vertex", "topk_k": 16,
   "topk_scorer": "fraction_negative", "topk_pruner": "davis",
   "n_epochs": 30, "lr": 0.01}}
```

The `hymeko_arch` and `hymeko_training` fields are the *provenance trail* — every result row knows which HyMeKo description produced it.

5.2 Sweep mode

A 2×2 micro-sweep on Bitcoin Alpha (`m` ∈ {16, 64} × `pruner` ∈ {none, davis}) ran end-to-end in 2 minutes, with one JSONL row per cell:

m	pruner	AUC	note
16	none	0.7930	—
16	davis	0.7931	davis kills ≈ 0 triads here
64	none	0.8481	+5 pp at ∼ 2× fwd cost
64	davis	0.8481	—

6 What this gives us

(1) **One source of truth.** The architecture, training, and tuning are now in version-controlled `.hymeko` files instead of hand-edited `run_*.py` call sites.

(2) **Strategy/command pattern is a config flip.** Cycle enumeration is a hyperedge with a `pruner` field; changing `"none"` → `"davis"` → `"balance"` requires zero code edits.

(3) **Provenance.** Every trained checkpoint knows the exact `.hymeko` that produced it.

(4) **Three tuning expressions, one driver.** Grid, GA, and P-graph axiom-feasibility all read the same arch+training pair; only the policy file changes.

(5) **Reuses production code.** The driver wraps `run_final_cell.cell_signed_graph` and the existing cycle enumerator — no parallel implementation to drift out of sync.

7 What's left

- **GA driver loop** is currently scaffolded; the `<sweep_policy kind="genetic">` branch in the driver needs to call `run_hsikan_genetic.py`'s evaluation function in place of the grid Cartesian product.
- **MSG/SSG/ABB driver branch** should call `hymeko_pgraph::abb_solve` on the parsed P-graph and emit the optimum architecture as a synthesised `arch_*.hymeko`.
- **Template-side codegen** for HSiKAN is already wired in `transforms/torch_dataflow/template.py` (Tier-3 `signedkan_layer` / `walk_layer` / etc.); a `queries.hymeko`-style query for our new `<dataflow>` shape would close the codegen loop and let `hymeko compile` emit a `torch.nn.Module` class directly from `arch_mixed_k34.hymeko`.
- **Translator-emit-back** so the driver could write a `.hymeko` for the best-found cell, completing the round-trip.